

Q	#1	Q	#2
What is the ISTQB definition of software testing?		Why is exhaustive testing impossible?	
Q	#3	Q	#4
List and briefly explain the 7 Test Principles.		What does Principle 4 (Defect Clustering) mean for test prioritisation?	
Q	#5	Q	#6
What is the Pesticide Paradox and how do you deal with it?		What is the cost-of-defects curve and why does it matter?	
Q	#7	Q	#8
How much testing is enough?		What are the 4 phases of the fundamental test process?	
Q	#9	Q	#10
What is the test basis?		What qualities does a good tester need?	
Q	#11	Q	#12
Describe the independence spectrum in testing (4 levels).		What is the difference between Error, Defect, and Failure?	

A Why is exhaustive testing impossible? #2	A What is the ISTQB definition of software testing? #1
The number of possible inputs, states, and paths through any non-trivial system is astronomically large. Instead, use risk and priorities to decide where to focus effort.	The process of all software life-cycle activities, both static and dynamic, concerned with planning, preparation, and evaluation of software products to determine that they satisfy specified requirements, demonstrate fitness for purpose, and detect defects.
A What does Principle 4 (Defect Clustering) mean for test prioritisation? #4	A List and briefly explain the **7 Test Principles**. #3
Defects pile up in a small fraction of the codebase (Pareto: ~80% of issues in ~20% of code). Focus test effort on those high-risk modules rather than spreading it evenly.	1. Testing shows presence of defects — never their absence. 2. Exhaustive testing is impossible — use risk + priorities. 3. Early testing saves money — fixing a defect in design is far cheaper than in production. 4. Defect clustering — 80% of defects come from 20% of code (Pareto). 5. Pesticide paradox — the same tests stop finding new bugs; tests must evolve. 6. Testing is context-dependent — safety-critical ≠ a marketing website. 7. Absence-of-errors fallacy — bug-free software is useless if it doesn't meet user needs.
A What is the **cost-of-defects curve** and why does it matter? #6	A What is the **Pesticide Paradox** and how do you deal with it? #5
The cost of fixing a defect rises sharply the later it is found. Fixing a bug in requirements costs orders of magnitude less than fixing it in production. This is the primary motivation for early testing.	Running the same test suite over and over eventually stops finding new bugs because the code has been patched for those specific cases. Solution: regularly review and add new tests, and update the test suite to cover different areas.
A What are the **4 phases** of the fundamental test process? #8	A How much testing is enough? #7
1. Plan & control — who, what, why, when, where; adjust as new info arrives. 2. Analysis & design — identify test conditions from the test basis; design test cases, data, environments. 3. Implementation & execution — group tests into scripts, prioritise, automate, execute, compare results, log. 4. Test completion (closure) — evaluate vs objectives, write test summary report, archive testware.	Depends on risk (technical + business) and project constraints (time, budget). There is no universal answer — testing should produce enough information to make good decisions.
A What qualities does a good tester need? #10	A What is the **test basis**? #9
Curiosity, professional pessimism, strong communication skills, skill at error guessing, and the ability to report defects constructively (fact-focused, not personal).	The information from which tests are derived — requirements, user stories, architecture, design documents, interfaces, risk reports.
A What is the difference between **Error**, **Defect**, and **Failure**? #12	A Describe the **independence spectrum** in testing (4 levels). #11
• Error / Mistake — a human action that produced an incorrect result (root cause). • Defect / Bug / Fault — the flaw in the code or design caused by the error. • Failure — the observable incorrect behaviour when the defect is executed at runtime.	1. Programmer tests their own code. (least independent) 2. Tester in the same team as developers. 3. Dedicated test team within the organisation. 4. External test organisation. (most independent) More independence = less bias, fresh perspective. Less independence = faster feedback, less communication overhead.

Q#13	Q#14
What is debugging, and how does it differ from testing?	Draw the V-model and label each level.
Q#15	Q#16
What is Verification vs Validation?	What is Component (Unit) testing?
Q#17	Q#18
What is a Stub, and what is a Driver? (and which calls which?)	What is Integration testing?
Q#19	Q#20
What is System testing?	What is Acceptance testing? Name four types.
Q#21	Q#22
Compare Confirmation testing (re-testing) vs Regression testing.	Give four examples of non-functional testing.
Q#23	Q#24
What are the three triggers for Maintenance testing?	What is Impact analysis in maintenance testing?
Q#25	Q#26
Why does agile development increase the importance of automated regression?	What is the difference between static and dynamic testing?

A Draw the V-model and label each level. #14 User requirements System requirements Global design Detailed design Implementation Acceptance testing System testing Integration testing Component (unit) testing Each test level is designed as soon as the corresponding development artefact exists.	A What is debugging, and how does it differ from testing? #13 Debugging is the developer-side process of finding and removing defects after a failure is observed. Testing finds and reports failures. Testers don't fix; developers do.
A What is Component (Unit) testing? #16 Tests individual modules, classes, or methods in isolation. Done by the programmer. Uses stubs and drivers to replace missing components. TDD lives at this level.	A What is Verification vs Validation? #15 • Verification — "Are we building the thing right?" Checks the product matches its specification. Happens throughout development. • Validation — "Are we building the right thing?" Checks the product meets actual user needs. Happens once the user/customer is involved (acceptance testing).
A What is Integration testing? #18 Tests the interfaces and interactions between components or systems. Basis: design, architecture, data flow, workflows. Tests two varieties: • Component integration — interactions after unit testing. • System integration — interactions with other external systems.	A What is a Stub, and what is a Driver? (and which calls which?) #17 • Stub — a simplified simulator called by the component under test (simulates a dependency called downward). • Driver — a simplified simulator that calls into the component under test (simulates a caller from above). Memory aid: the driver drives the test; the stub stands in for a dependency.
A What is Acceptance testing? Name four types. #20 The final level — checks the system meets user/business needs (validation). Types: 1. UAT (User Acceptance Testing) 2. Alpha testing — in-house, by intended users 3. Beta testing — real users in their real environment 4. Operational acceptance — operational readiness (backups, recovery)	A What is System testing? #19 Tests the complete, integrated system against its specified requirements. Covers both functional and non-functional aspects. Performed in a near-production environment.
A Give four examples of non-functional testing. #22 Performance (load/stress/scalability), Usability, Security, Reliability/Recoverability, Portability, Interoperability, Accessibility, Maintainability.	A Compare Confirmation testing (re-testing) vs Regression testing. #21 • Confirmation testing — re-runs the specific test that <i>failed</i> to confirm the defect has been fixed. • Regression testing — re-runs the <i>previously passing</i> tests to confirm the fix didn't break anything else. Heavily automated.
A What is Impact analysis in maintenance testing? #24 Before testing a change, assess: what parts of the system are affected, what side effects the change may have, and how much regression testing is therefore needed. Hard when documentation is missing or original developers are gone.	A What are the three triggers for Maintenance testing? #23 1. Modification — bug fix or new feature. 2. Migration — moving to a new platform, data format, or infrastructure. 3. Retirement — archiving data and decommissioning the system.
A What is the difference between static and dynamic testing? #26 • Static — examines artefacts without executing them (reviews, walkthroughs, inspections, static analysis). Finds defects in requirements, design, code structure. • Dynamic — executes the software to observe failures at runtime (unit tests, system tests, BVA).	A Why does agile development increase the importance of automated regression? #25 In agile, every iteration may change or add features. Without automated regression, re-verifying all previous functionality each sprint is too slow and expensive to be practical.

Q	#27	Q	#28
List the 6 phases of a formal review.		What are the four types of reviews in order of increasing formality?	
Q	#29	Q	#30
What are the roles in a formal review?		What are the three defect severity levels in a review?	
Q	#31	Q	#32
What is Static Analysis and what does it find?		What is cyclomatic complexity?	
Q	#33	Q	#34
What is the difference between control flow and data flow analysis?		Name four defects that static analysis finds better than dynamic testing.	
Q	#35	Q	#36
What is TDD?		Describe the Red → Green → Refactor cycle.	
Q	#37	Q	#38
What are the advantages of TDD?		What are common TDD pitfalls?	

A What are the four types of reviews in order of increasing formality? #28	A List the 6 phases of a formal review. #27
1. Informal review — no documentation, pair check, fast. 2. Walkthrough — author presents; educational; no mandatory pre-prep. 3. Technical review — discussion meeting; technical decisions + defect detection. 4. Inspection — most formal; pre-meeting prep, checklists, trained roles, metrics, formal log.	1. Planning — scope, document size, assign reviewer roles. 2. Kick-off (optional) — entry/exit criteria, role assignment; studies show kick-offs find ~70% more major defects. 3. Preparation — reviewers read individually, build checklists, log defects. 4. Review meeting — log defects (focus: log, don't debate); aim 1–2 defects/minute. 5. Rework — author fixes (or argues why not). 6. Follow-up — moderator confirms all defects addressed; collects metrics.
A What are the three defect severity levels in a review? #30	A What are the roles in a formal review? #29
• Critical — will damage or invalidate other documents/work products. • Major — could damage other documents. • Minor — cosmetic only.	• Moderator — runs the process, distributes work, ensures quality. • Author — created the document under review. • Scribe — logs defects during the meeting. • Reviewers (checkers/inspectors) — find defects. • Manager (sometimes) — allocates resources, decides on reviews.
A What is cyclomatic complexity? #32	A What is Static Analysis and what does it find? #31
A code metric that counts the number of independent paths through a piece of code. High cyclomatic complexity → hard to test, likely defect-prone. Can be used to identify hot-spots for review.	Tool-based examination of code or design artefacts without executing them. Finds: coding standard violations, code metrics hot-spots (cyclomatic complexity, nesting depth), unreachable code, uninitialised variables, inconsistent interfaces, data flow anomalies.
A Name four defects that static analysis finds better than dynamic testing. #34	A What is the difference between control flow and data flow analysis? #33
Dead/unreachable code, uninitialised variables, off-spec interfaces, cyclomatic complexity hot-spots, unused variables.	• Control flow — the sequence of execution through code. Can detect unreachable code and deep nesting. • Data flow — tracks which data items are read, written, or initialised. Can detect uninitialised variables and anomalies like "written but never read."
A Describe the Red → Green → Refactor cycle. #36	A What is TDD? #35
• Red — write a small test for behaviour that doesn't yet exist; run it; see it fail (confirms the test is testing something real). • Green — write the minimum production code to make the test pass. Don't over-engineer. • Refactor — improve names, remove duplication, simplify — without changing behaviour. All tests must still pass.	A development practice where you write a failing test first, then write the minimum code to make it pass, then refactor — without changing behaviour. Applied mostly at the unit/component level.
A What are common TDD pitfalls? #38	A What are the advantages of TDD? #37
• Steps too big — test covers too much, hard to make pass. • Writing production code first "to save time." • Tests too implementation-dependent — break on any refactor. • Insufficient refactoring — code is messy and brittle. • "I trust that it works" — rarely true for edge cases.	Fast feedback (seconds to know if something broke), simpler design (only what's needed), safer changes (strong test suite catches regressions), testable code from day one, tests act as executable documentation.

Q #39	Q #40
What do the JUnit annotations <code>@Test</code> , <code>@BeforeEach</code> , <code>@AfterEach</code> , <code>@BeforeAll</code> , <code>@AfterAll</code> do?	What is the difference between <code>assertEquals</code> and <code>assertAll</code> in JUnit?
Q #41	Q #42
What are the Leap Year rules (used as a TDD example)?	What makes a good JUnit test name?
Q #43	Q #44
Why must a test verify only one thing?	What is the difference between a test condition, a test case, and a test procedure?
Q #45	Q #46
What is traceability in testing, and what are horizontal vs vertical traceability?	What are the three families of test design techniques?
Q #47	Q #48
Explain Equivalence Partitioning (EP).	Explain Boundary Value Analysis (BVA).
Q #49	Q #50
When do you use Decision Table Testing?	What is State Transition Testing?
Q #51	Q #52
What is Use Case Testing and at which levels is it applied?	What is Statement Coverage?

A What is the difference between <code>`assertEquals`</code> and <code>`assertAll`</code> in JUnit? #40	A What do the <code>**JUnit**</code> annotations <code>`@Test`</code>, <code>`@BeforeEach`</code>, <code>`@AfterEach`</code>, <code>`@Before`</code>, and <code>`@After`</code> do? #39
• <code>assertEquals(expected, actual)</code> — fails fast on the first mismatch. • <code>assertAll(...)</code> — groups multiple assertions; reports all failures, not just the first one.	• <code>@Test</code> — marks the method as a test case. • <code>@BeforeEach</code> — runs before every test method. • <code>@AfterEach</code> — runs after every test method. • <code>@BeforeAll</code> — runs once before all tests in the class. • <code>@AfterAll</code> — runs once after all tests in the class.
A What makes a <code>**good JUnit test name**</code>? #42	A What are the <code>**Leap Year rules**</code> (used as a TDD example)? #41
It describes the scenario and the expected result. Bad: <code>test1</code>. Good: <code>leapYear_DivisibleBy400_IsLeap</code>.	• Divisible by 4 → leap year. • Divisible by 100 → not a leap year. • Divisible by 400 → is a leap year (overrides the 100 rule). Order of TDD: start with the smallest rule ($\div 4 \rightarrow$ leap), then add exceptions one test at a time.
A What is the difference between a <code>**test condition**</code>, a <code>**test case**</code>, and a <code>**test procedure**</code>? #44	A Why must a test verify <code>**only one thing**</code>? #43
• Test condition — anything that <i>could</i> be tested (a behaviour, a rule, a value). • Test case — a concrete input + expected result + preconditions + postconditions. • Test procedure / script — an ordered, executable sequence of test cases.	So that when it fails, the reason is immediately obvious. A test covering multiple behaviours can fail for multiple reasons, making diagnosis much harder.
A What are the three families of <code>**test design techniques**</code>? #46	A What is <code>**traceability**</code> in testing, and what are horizontal vs vertical traceability? #45
1. Black-box (specification-based) — test what the system should do; no knowledge of internals. 2. White-box (structure-based) — test using knowledge of internal code structure for coverage. 3. Experience-based — use tester's intuition + skill (error guessing, exploratory).	Traceability = linking a test back to the requirement or test basis item it covers. • Horizontal — across documents at the same level (e.g., test case ↔ requirement at system level). • Vertical — across levels (e.g., acceptance test ↔ user requirement ↔ component test).
A Explain <code>**Boundary Value Analysis (BVA)**</code>. #48	A Explain <code>**Equivalence Partitioning (EP)**</code>. #47
Defects cluster at the edges of equivalence partitions. For each boundary, test: the boundary value itself, one value just inside, one value just outside. For 18–65: test 17, 18, 19 (lower bound) and 64, 65, 66 (upper bound).	Group inputs that should be treated the same way by the system into partitions. Test one value per partition. Example: age 18–65 valid → partitions are <18 (invalid), 18–65 (valid), >65 (invalid). Test one value from each (e.g., 10, 30, 80).
A What is <code>**State Transition Testing**</code>? #50	A When do you use <code>**Decision Table Testing**</code>? #49
Models the system as a finite state machine — states, events (transitions), guard conditions, actions. A state table lists all states × events; cells show the next state. Useful for finding invalid transitions. Coverage options: every state, every valid transition, every pair of valid transitions, every invalid transition.	When combinations of input conditions drive different outcomes (business rules). Each column is one rule (unique combination of condition T/F values), and the bottom rows show the resulting actions. Strengths: complete coverage, flushes out missing or contradictory rules.
A What is <code>**Statement Coverage**</code>? #52	A What is <code>**Use Case Testing**</code> and at which levels is it applied? #51
Has every executable statement been executed at least once? Easy to achieve high numbers but does not guarantee all branch logic is tested.	Tests actor–system interaction flows — each use case has preconditions, main flow, alternative flows, postconditions. Great at finding integration defects and real-world workflow issues. Mostly applied at system and acceptance levels.

Q	#53	Q	#54
What is Decision (Branch) Coverage, and how does it compare to statement coverage?		What is Error Guessing?	
Q	#55	Q	#56
How do you choose which test design technique to use?		What is test automation?	
Q	#57	Q	#58
Describe the test automation pyramid (3 layers).		Give 5 benefits of test automation.	
Q	#59	Q	#60
Give 5 risks or downsides of test automation.		When should you NOT automate?	
Q	#61	Q	#62
Compare Capture & Playback, Structured Scripting, and Model-based Scripting.		What is Playwright, and what are its key features?	
Q	#63	Q	#64
What is synthetic test data?		What are the key challenges of test automation in agile?	
Q	#65	Q	#66
List 8 items found in an IEEE 829 test plan.		What are entry criteria vs exit criteria?	

A What is Error Guessing? #54	A What is Decision (Branch) Coverage, and how does it compare to statement coverage? #53
An experience-based technique where the tester predicts where defects are likely to hide based on domain knowledge and past experience. Common targets: off-by-one errors, null inputs, empty lists, encoding issues, time zones. Best supported by maintaining a defect/error list.	For every if, while, for, case — has each outcome (True and False) been tested? Stronger than statement coverage. 100% decision coverage implies 100% statement coverage (but not vice versa).
A What is test automation? #56	A How do you choose which test design technique to use? #55
Use of specialised software, separate from the software under test, to automatically execute tests, compare results, and report outcomes.	No single recipe. Consider: test basis available, risk profile and regulatory context, tester skill, time/cost, existing models (state machines, decision tables), and system complexity. Often combine several: BVA + decision tables + exploratory is a standard mix.
A Give 5 benefits of test automation. #58	A Describe the test automation pyramid (3 layers). #57
Increased efficiency (faster, more frequent runs), improved accuracy (no human fatigue/typos), reusability across versions/browsers/environments, enables continuous testing in CI/CD, tests act as executable documentation/reports.	[GUI / E2E] ← few, slow, high-level confidence [Integration] [Unit / Component] ← many, fast, low-level confidence Most tests should live at the bottom. Unit/integration tests are written by developers; GUI/E2E is owned by test automation engineers.
A When should you NOT automate? #60	A Give 5 risks or downsides of test automation. #59
Tests that can't be automated (subjective UX/usability), tests with very low risk or that run only once, exploratory work, systems about to be retired, systems that change drastically/often. Rule of thumb: don't spend more time automating than the manual effort would cost.	High initial investment, ongoing maintenance burden (tests rot when UI changes), technical skills required, tool licences and infrastructure costs, false alarms (flakiness lowers trust in the suite), can distract from real testing objectives.
A What is Playwright, and what are its key features? #62	A Compare Capture & Playback, Structured Scripting, and Model-based Scripting. #61
A Microsoft (2020) open-source, JavaScript/TypeScript end-to-end testing framework for web and API testing. Features: cross-browser (Chromium, Firefox, WebKit), multiple language bindings, parallel execution, network manipulation, built-in screenshots/video, easy CI integration.	Approach Abstraction Skill needed Maintenance Example Capture & Playback None Low High (brittle) Selenium IDE Structured Scripting Yes (Page Objects, functions) Medium Medium Playwright, Cypress Model-based Yes (system model) High Low Tosca, SpecFlow
A What are the key challenges of test automation in agile? #64	A What is synthetic test data? #63
Rapid requirement changes, maintaining speed under tight sprint cycles, balancing customer value vs adequate coverage, tests growing in complexity as features grow, cross-team collaboration, keeping the automation suite from becoming a maintenance burden.	Artificially created data that mimics real-world data, used in non-production environments. Sources: build it yourself, fetch from other teams, fetch from external systems (e.g., Norwegian <i>Tenor testdata</i> service).
A What are entry criteria vs exit criteria? #66	A List 8 items found in an IEEE 829 test plan. #65
• Entry criteria — conditions that must be met before testing can start (e.g., build available, smoke test passes, environment ready). • Exit criteria — conditions that must be met before testing can stop (e.g., coverage targets met, defect rate under threshold, time/budget exhausted).	Test plan identifier, Introduction/scope, Test objectives, Test items, Features to/not to be tested, Approach/strategy, Pass/fail criteria, Suspension/resumption criteria, Test deliverables, Testing tasks/schedule, Environmental needs, Responsibilities/staffing/training, Risks and contingencies, Approvals.

Q	#67	Q	#68
What is the difference between project risk and product risk?		What are the four responses to risk?	
Q	#69	Q	#70
How is risk analysis done?		What does a defect report (IEEE 829) contain?	
Q	#71	Q	#72
What is the defect lifecycle?		What is Configuration Management and why does it matter for testing?	
Q	#73	Q	#74
What is the difference between test progress monitoring and test control?		List the six tool categories (ISTQB hexagon).	
Q	#75	Q	#76
What are the four purposes of using test tools?		Name 3 benefits and 3 risks of test tools.	
Q	#77	Q	#78
What is the probe effect?		What is the difference between the tester's role and the tool user's role?	

A What are the four responses to risk? #68	A What is the difference between project risk and product risk? #67
1. Mitigate — reduce probability or impact (e.g., extra testing, code review). 2. Contingency — plan what to do if it occurs. 3. Transfer — push the risk elsewhere (insurance, vendor contract). 4. Ignore — accept it if cost/impact is low.	• Project risk — threats to the project's ability to deliver (organisational, technical, supplier issues). • Product risk — threats that the product will fail to satisfy users or cause harm. Example of project risk: staff shortages. Example of product risk: a safety-critical calculation is wrong.
A What does a defect report (IEEE 829) contain? #70	A How is risk analysis done? #69
Description of the defect, steps to reproduce, impact/severity, classification (scope, severity, priority), resources involved, root cause (captured by the programmer who fixed it), conclusions and recommendations, lifecycle status.	Rate each risk item 0–10 on probability × impact. Educated guesses, refined at project milestones. In a V-model project: at requirements phase, end of design, during implementation, and throughout testing. Produces a risk-based testing priority list.
A What is Configuration Management and why does it matter for testing? #72	A What is the defect lifecycle? #71
The process of identifying and tracking every change to test items and testware (code, test scripts, test data, environments). Ensures a defect found in build X can be reproduced against the exact same version X. Provides traceability for audits.	Opened → Assigned → Fixed → Verified → Closed. May also be: Deferred (fix later) or Rejected (not a real defect).
A List the six tool categories (ISTQB hexagon). #74	A What is the difference between test progress monitoring and test control? #73
1. Management (test mgmt, requirements mgmt, defect tracking, configuration mgmt) 2. Static testing (review tools, static analysis, modelling tools) 3. Test specification (test design tools, test data preparation) 4. Execution and logging (execution, unit testing, comparators, coverage, security) 5. Performance and monitoring (dynamic analysis, monitoring, load/stress) 6. Specific application areas (domain-specific: mobile, embedded, web)	• Monitoring — collecting data on how testing is proceeding (tests run/passed/failed, defects found, coverage %). Provides feedback/overview. • Control — acting on that data to adjust: re-prioritise areas, adjust schedule/scope, tighten/loosen entry/exit criteria.
A Name 3 benefits and 3 risks of test tools. #76	A What are the four purposes of using test tools? #75
Benefits: reduced repetitive work, greater consistency/repeatability, objective assessment (no human bias), easier access to information (dashboards). Risks: unrealistic expectations (won't solve process problems), underestimating introduction costs (training, integration), underestimating asset maintenance (scripts rot), over-reliance replacing good manual testing.	1. Improve efficiency of test activities (automate repetitive tasks). 2. Automate things needing significant manual resources (e.g., static analysis at scale). 3. Automate what cannot be done manually (e.g., load testing thousands of users). 4. Increase reliability (consistent comparisons, simulation of complex behaviour).
A What is the difference between the tester's role and the tool user's role? #78	A What is the probe effect? #77
• Tester — decides <i>what</i> to test, <i>what</i> test cases to design, <i>how</i> to prioritise. Owns the testing strategy. • Tool user — decides <i>how</i> to operate the tool effectively, how to extract maximum benefit, how to maintain and scale it. Both roles are needed. A tool is only as useful as the testing strategy behind it.	The act of measuring (e.g., adding a performance monitoring probe) changes the behaviour being measured. An intrusive tool is one whose presence affects the system under test.

Q	#79	Q	#80
How should you introduce a test tool into an organisation?		Which test level is most heavily automated, and which is least?	
Q	#81	Q	#82
What are the four ways testers use LLMs (CT-GenAI)?		Why does LLM-generated code need extra testing scrutiny?	
Q	#83	Q	#84
What is hallucination in the context of LLM testing?		What is the key ISTQB principle about LLM-generated output?	
Q	#85	Q	#86
What is the difference between CT-AI and CT-GenAI?		What did the HumanEval TDD study show about LLMs?	
Q	#87	Q	#88
What is the difference between SAST and DAST?		How do classic testing techniques map to vulnerability research?	
Q	#89	Q	#90
Define exploratory testing.		What is the difference between Wrong and Weird in testing?	
Q	#91	Q	#92
Why is exploratory testing classified as both dynamic and experience-based?		What is the COUNT heuristic?	

A Which test level is most heavily automated, and which is least? #80	A How should you introduce a test tool into an organisation? #79
Unit testing is most heavily automated (fast, cheap, isolated). Acceptance testing is least automated (often kept manual; requires user/business context).	Run a pilot project (small, representative scope). Define clear objectives (what does success look like). Ensure the process is in place first (automating chaos = chaos). Plan for training and ongoing support. Track ROI. Adapt usage as the team learns.
A Why does LLM-generated code need extra testing scrutiny? #82	A What are the four ways testers use LLMs (CT-GenAI)? #81
• Non-determinism — the same prompt may produce different code each time. • Subtle logic errors — code compiles and looks right but fails on edge cases. • Security blind spots — may produce injection-prone code, hardcoded secrets, weak crypto. • Outdated patterns — training data may include deprecated APIs or libraries with CVEs.	1. Test case generation — from requirements, user stories, or API specs via structured prompts. 2. Test data synthesis — realistic edge cases, boundary values, equivalence partitions. 3. Test script drafting — Cypress, Playwright, pytest scaffolding from natural language. 4. Defect prediction — analysing code changes to prioritise where to test.
A What is the key ISTQB principle about LLM-generated output? #84	A What is hallucination in the context of LLM testing? #83
The tester remains accountable for evaluating and validating all LLM-generated output. The LLM is a tool; the human owns the quality judgement.	An LLM producing plausible but incorrect test cases — tests that look reasonable but do not match the actual specification. The tester must always verify LLM output against the real requirements.
A What did the HumanEval TDD study show about LLMs? #86	A What is the difference between CT-AI and CT-GenAI? #85
Three pipelines (Iterative TDD, Batch, NoTDD) were tested on 164 Python tasks. Iterative TDD (write one test → run → implement → next test) usually outperformed NoTDD (direct prompting without tests). Conclusion: test-first feedback loops are useful even for LLMs.	• CT-AI (2021) — Testing of AI systems: data quality, bias, non-determinism, ML metrics (how to test an AI product). • CT-GenAI (2025) — Testing with generative AI: prompt engineering, LLM evaluation, RAG, integration (how to use GenAI as a testing tool).
A How do classic testing techniques map to vulnerability research? #88	A What is the difference between SAST and DAST? #87
• Boundary Value Analysis → finds buffer overflows. • Equivalence Partitioning → finds input-validation gaps. • Control-flow testing → reveals authentication bypasses. Same foundational techniques, different stakes.	• SAST — Static Application Security Testing: analyses source code/binaries without running the app. • DAST — Dynamic Application Security Testing: attacks the running application to find vulnerabilities.
A What is the difference between Wrong and Weird in testing? #90	A Define exploratory testing. #89
• Wrong — violates a specified rule or contract (caught by scripted tests). • Weird — violates an unstated expectation, taste, or rhythm (only a human is likely to notice). Both matter. Examples of "weird": an error message that is technically correct but sounds condescending; a button in an unexpected location.	Simultaneous learning, test design, and test execution — guided by the tester, not a script. It focuses on discovery and relies on the tester's domain knowledge and curiosity.
A What is the COUNT heuristic? #92	A Why is exploratory testing classified as both dynamic and experience-based? #91
Test with zero, one, many, too many, and too few items. Example: search returning no results (0), one result (1), many results (n), an overloaded result set (too many), a system that should show results but shows none (too few).	Dynamic because it executes the software. Experience-based because it relies on the tester's intuition and domain knowledge to guide what to explore.

Q	#93	Q	#94
What is the Goldilocks heuristic?		What is the CRUD heuristic?	
Q	#95	Q	#96
What does RCRCRC stand for, and when is it used?		What are trigger heuristics in exploratory testing?	
Q	#97	Q	#98
What is the difference between a mnemonic and a heuristic?		What is a test charter in exploratory testing?	
Q	#99	Q	#100
What are the 5 steps for doing exploratory testing?		Name the 9 bug taxonomy categories.	
Q	#101	Q	#102
When does exploratory testing shine (best-fit scenarios)?		Why does context matter so much in exploratory testing?	
Q	#103	Q	#104
Test coverage — definition.		Testware — definition.	
Q	#105	Q	#106
Test strategy — definition.		Incident — definition.	

A What is the CRUD heuristic? #94	A What is the Goldilocks heuristic? #93
For any entity, test: Create (new + duplicate), Read (existing + non-existent), Update (existing + non-existent), Delete (existing + non-existent). Covers the full lifecycle of data.	Test with too big, too small, and just right inputs. • Too big: overload a text field. • Too small: leave it empty. • Just right: the happy-case input.
A What are trigger heuristics in exploratory testing? #96	A What does RCRCRC stand for, and when is it used? #95
Use your own emotional reactions as signals: • Impatience → investigate delays. • Confusion → investigate counterintuitive behaviour. • Surprise → an assumption has been violated. • Discomfort → something feels off → investigate it.	A regression scoping mnemonic: • Recent — newly added code • Core — key functionality that must always work • Risky — areas relying on other services • Config — areas affected by configuration/environment • Repaired — code changed during a recent bug fix • Chronic — areas that frequently break
A What is a test charter in exploratory testing? #98	A What is the difference between a mnemonic and a heuristic? #97
A short mission statement for an exploratory session: • Target — what is being explored • Resources — tools, data, time budget (typically time-boxed, e.g., 90 minutes) • Information goals — what you hope to learn	• Mnemonic — a memory aid (e.g., CRUD, RCRCRC). • Heuristic — a problem-solving rule of thumb that often works but is not guaranteed to work in all cases.
A Name the 9 bug taxonomy categories. #100	A What are the 5 steps for doing exploratory testing? #99
Input validation, Boundary, Calculation, Usability, Security, Logic, Compatibility, Syntax, Performance.	1. Develop a bug classification (taxonomy from past projects + root cause analysis). 2. Understand the system under test (type, functionality, users). 3. Choose a heuristic (CRUD, COUNT, Goldilocks, RCRCRC, trigger emotions, etc.). 4. Create a test charter (target, resources, goals). 5. Continuously assess findings: take notes, adapt next session, file bugs, update charter.
A Why does context matter so much in exploratory testing? #102	A When does exploratory testing shine (best-fit scenarios)? #101
The same behaviour can produce very different user experiences depending on who the user is. Example: a 200ms delay is trivial for one user but unacceptable for a nurse on night shift under time pressure. Context is lived, not declared in a doc.	Poor or missing documentation, limited time, unfamiliar/new product, areas where scripted tests pass but users still complain, high-risk usability/UX flows, after an automated regression run to find what scripted tests missed.
A Testware — definition. #104	A Test coverage — definition. #103
All artefacts produced during testing: test cases, test scripts, test data, test environments, test plans, test reports.	The percentage of identified test items (statements, branches, partitions, etc.) that have been exercised by the test suite. 100% does not mean fully tested.
A Incident — definition. #106	A Test strategy — definition. #105
An event during testing that requires investigation (may or may not turn out to be a defect).	A high-level description of the test levels and techniques to be used on a project; derived from the test policy or from risk analysis.

Q #107	Q #108
Severity vs Priority (defect management).	What is COTS?
Q #109	
What is the risk formula?	

A What is COTS? #108 Commercial Off-the-Shelf software — pre-built software products bought rather than developed in-house. Needs its own acceptance and integration testing.	A Severity vs Priority (defect management). #107 • Severity — the technical impact of the defect on the system. • Priority — the business urgency of fixing the defect. A cosmetic bug on a login page may have low severity but high priority (high visibility). A catastrophic crash in a rarely used feature may have high severity but lower priority.
	A What is the risk formula? #109 Risk = Probability × Impact (both rated 0–10).